Sibyl: Setup Guide

Sibyl

14 October 2026

CONTENTS

Setting up Sibyl 4

Prerequisites 5

Software 5

Per operating system 6

Hardware 7

Ports 7

Corporate TLS-inspection proxies 8

Demo walkthrough 9

Real deployment walkthrough 10

1. Start it 10

2. Trust the local certificate authority 11

3. Sign in 11

4. Create keys for your agents 12

5. Connect a real agent 12

6. Write a first Action Gate policy 15

7. Register an external monitor 17

8. Enable Sentinel 18

Capture policy 18

Production hardening 18

OIDC provider 18

TLS certificates 20

Secrets and rotation 21

Backups and restore drill 21

Upgrades and migrations 22

Resource sizing 22

Production checklist 22

Codespaces 23

Troubleshooting 24

Reference 27

Setting up Sibyl

Sibyl is a self-hosted control plane for an organisation's AI agents. It watches every agent, stops unauthorised actions before they run (the Action Gate), moves traffic when a model or provider fails, and keeps an audit trail an auditor can verify offline.

There are two ways to run it:

	Demo	**Real deployment**
What you get	The console with dummy data and a scripted outage scenario; every screen clickable	The full stack for your own agents: Postgres, API, ingest, worker, gateway, console, Caddy TLS, OIDC sign-in; no simulator
Needs	Node.js only	Docker with Compose
In the browser	Open in Codespaces (demo)	Open in Codespaces (real)
On your machine	`make demo`	`make up`
Short guide	DEMO.md	REAL.md

A third option, `make demo-stack`, runs the full stack in Docker seeded with the simulator's demo world (three tenants, 20 agents, simulated providers); it is what the quickstart walks through.

Contents: Prerequisites · Corporate TLS-inspection proxies · Demo walkthrough · Real deployment walkthrough · Production hardening · Codespaces · Troubleshooting · Reference

Prerequisites

SOFTWARE

Tool	Version	Demo	Real	Check
git	any recent	yes	yes	`git --version`
make	GNU make 3.81+ (the macOS one is fine)	yes	yes	`make --version`
Node.js	20.18 or later (22 LTS recommended)	yes	for console development only	`node --version`
Docker Engine	24 or later (Docker Desktop 4.27+, or Colima 0.6+)	no	yes	`docker version`
Docker Compose	v2, **2.24.4 or later**	no	yes	`docker compose version`
openssl	1.1.1 or 3.x	no	yes	`openssl version`
htpasswd	any (bcrypt for the admin password)	no	yes	`htpasswd -h`
uv	0.5 or later (the repository uses 0.13)	no	for the SDK, CLI and tests	`uv --version`
Python	3.13 (uv installs it for you)	no	for the SDK, CLI and tests	`uv python find 3.13`
jq, curl	any	no	handy for the API examples	`jq --version`

Everything Sibyl itself runs (Python, Postgres 17, Caddy, dex, Node for the console) is inside the containers. Python and uv on the host are only for your agents, the `sibyl` CLI and development.

PER OPERATING SYSTEM

macOS (Intel or Apple Silicon). `xcode-select --install` gives git, make, openssl and htpasswd. Then `brew install node@22 uv jq` and one Docker runtime:

- **Docker Desktop**: give it at least 4 CPUs and 8 GB of memory (Settings → Resources).
- **Colima** (free, no licence question): `brew install colima docker docker-compose`, then `colima start --cpu 4 --memory 8 --disk 60 --vm-type vz --mount-type virtiofs`. Make the Compose plugin visible to `docker`: add `"cliPluginsExtraDirs": ["/opt/homebrew/lib/docker/cli-plugins"]` to `~/.docker/config.json`. Colima mounts only your home directory into its VM, and Compose bind-mounts Sibyl's secret files, so keep the repository under `$HOME`.

Apple Silicon builds and runs every image natively (`linux/arm64`); the released images are multi-arch.

Linux (Ubuntu 22.04/24.04, Debian 12, Fedora 40+). Install Docker Engine and the Compose plugin from Docker's repository (`docker-ce`, `docker-compose-plugin`; the distribution's `docker.io` is often too old for Compose 2.24.4), add yourself to the `docker` group, then: `sudo apt-get install -y make git openssl apache2-utils jq` (Fedora: `httpd-tools` for htpasswd), Node 22 from NodeSource or `nvm`, and uv with `curl -LsSf https://astral.sh/uv/install.sh | sh`.

Windows. Use WSL2 with Ubuntu 24.04 (`wsl --install -d Ubuntu-24.04`) and follow the Linux steps inside it. Run Docker either as Docker Desktop with the WSL2 backend (enable integration for the Ubuntu distribution) or as Docker Engine inside WSL2. Clone the repository inside the WSL file system (`~/src/sibyl`, not `/mnt/c/...`): bind mounts and file permissions on `/mnt/c` break the secret files and are slow. Open the console from the Windows browser at the same `https://localhost` URL.

HARDWARE

	Demo	Real (evaluation)	Real (production, one host)
CPU	2 cores	4 cores	8 cores
Memory	2 GB free	8 GB for Docker	16 GB
Disk	2 GB	20 GB (images about 4 GB)	100 GB+ SSD, sized by retention and payload capture

PORTS

Port (default)	What	Bound to
3000 (3001)	demo console (and its origin proxy target in Codespaces)	all interfaces
8443	console, API (`/api/*`), dex (`/dex/*`) over HTTPS	`SIBYL_BIND_ADDRESS` (127.0.0.1)
8444	gateway (`/v1/*`, OpenAI / Anthropic / Gemini)	`SIBYL_BIND_ADDRESS`
8445	ingest (OTLP/HTTP, `sibyl-event/1`)	`SIBYL_BIND_ADDRESS`
5435	Postgres (real deployment; backups only)	127.0.0.1
5434, 8300, 18082	Postgres and the simulator of `make demo-stack`	127.0.0.1

The real deployment and `make demo-stack` both default to 8443-8445; run one at a time, or move one with `SIBYL_PORT_HTTPS`, `SIBYL_PORT_GATEWAY_TLS` and `SIBYL_PORT_INGEST_TLS`.

Corporate TLS-inspection proxies

Some networks (Netskope, Zscaler, Palo Alto, Blue Coat) re-sign every TLS connection with a corporate CA. Downloads then fail with `self-signed certificate in certificate chain`, `UnknownIssuer` or `CERTIFICATE_VERIFY_FAILED`. **Never turn certificate verification off.** Give each tool the corporate CA bundle (PEM) instead; ask IT for it, or export it from the system keychain. The examples use `~/.config/corp-ca.pem`.

Where	What to set
Docker image builds	`SIBYL_EXTRA_CA_FILE=~/.config/corp-ca.pem` (shell or `deploy/compose/.env.real`). Passed as an optional BuildKit secret into the dependency-download steps only; it never lands in an image layer. Unset, builds use the normal trust store, so the same files work on any network.
The gateway calling providers	`SIBYL_UPSTREAM_CA_HOST_FILE=~/.config/corp-ca.pem` (a bundle that *replaces* the container's trust store for provider TLS; include the public roots too if only some traffic is inspected). Without it the gateway answers `502 upstream_error` for real providers.
uv (Python)	`export UV_SYSTEM_CERTS=1` after adding the CA to the OS trust store (the Makefile and `ci/run.sh` set it), or `SSL_CERT_FILE=~/.config/corp-ca.pem` for uv only.
npm	`npm config set cafile ~/.config/corp-ca.pem`, or `export NODE_EXTRA_CA_CERTS=~/.config/corp-ca.pem`. (`console/.npmrc` names a CA file; npm ignores it where it does not exist.)
Bun, Node tools	`NODE_EXTRA_CA_CERTS=~/.config/corp-ca.pem`
Colima VM	copy the CA into the VM's trust store if `docker pull` fails: `colima ssh -- sudo sh -c 'cat >> /etc/ssl/certs/ca-certificates.crt' < ~/.config/corp-ca.pem && colima restart`
Your agents	the agent's own HTTP client trust (for example `SSL_CERT_FILE`, `REQUESTS_CA_BUNDLE`)

Note that `SSL_CERT_FILE` *replaces* the trust store for whatever reads it. Set it per process (for example `env SSL_CERT_FILE=... sibyl-run app.py`), not globally, or uv and pip stop reaching PyPI.

Demo walkthrough

```
git clone https://github.com/MK-JKC/sibyl.git && cd sibyl
make demo
```

The first run installs the console's dependencies and builds it (a minute or two), then serves http://localhost:3000. `PORT=3100 make demo` picks another port; `make demo-dev` runs `next dev` with hot reload instead. No Docker, database or network access is needed once it is built.

You land on **Fleet** as the **Platform Admin** in **Meridian Logistics**; a purple **Demo data** chip marks every screen. The scenario starts with the server: Anthropic latency climbs at 0:15, a hard outage at 0:30 turns seven tiles red, one provider incident opens instead of seven alerts, and tiers 1, 2 and 3 fail over to OpenAI with handoff packets in the queue. At 3:20 a tier-1 move waits for a four-eyes approval; at 10:00 Anthropic recovers and failback follows. Click the chip to replay, pause or skip ahead. Switch persona (Platform Admin, On-call Operator, Compliance Auditor) from the card at the bottom of the sidebar. `SIBYL_DEMO_ANCHOR=2026-10-10T03:30:00Z make demo` pins the scenario clock. Details: console/README.md.

`make demo-stack` runs the same story against the real services (`ops/demo/up.sh` plus the seed; stop it with `make demo-stack-down`). Follow the quickstart from step 3.

Real deployment walkthrough

1. START IT

```
git clone https://github.com/MK-JKC/sibyl.git && cd sibyl
# optional, before the first run: name the workspace and the admin
export SIBYL_TENANT_SLUG=acme SIBYL_TENANT_NAME="Acme Corp"
SIBYL_ADMIN_EMAIL=you@acme.example
make up
```

`make up` (`deploy/compose/real.sh up`) does, in order:

1. Writes `deploy/compose/.env.real` from `env/compose.env.example` and `real.env.example` (settings, no secrets), and generates every secret into `deploy/compose/env-real/secrets/` (directory 0700) with `init-env.sh`: database role passwords, the KEK, the key pepper, salts, the snapshot, ledger and audit-anchor signing keys, OIDC client secrets and the admin's password. Nothing secret is printed or committed.
2. Builds the control, gateway and console images (about 5-10 minutes the first time), or pulls the released ones when `SIBYL_IMAGE_TAG` is set.
3. Starts Postgres 17, creates the database roles (`bootstrap`, the only step that sees the Postgres superuser password), applies migrations, and registers the snapshot and audit-anchor signing keys.
4. Starts every service and waits until each is healthy.
5. Bootstraps the tenant: creates it (with its KEK-wrapped data keys), registers the OIDC issuer, invites the Platform Admin (role `owner`, shown as **Admin**), sets the tenant capture policy (`metadata` by default: no prompt or response content is stored; see capture policy), and mints the gateway's own keys straight into its secret files.
6. Prints the URLs and next steps.

Re-running `make up` is safe: it keeps secrets, data and keys, and applies new migrations.

```
Sibyl is up (project sibyl-real, tenant "acme").
  console   https://localhost:8443        (sign in to workspace "acme")
  api       https://localhost:8443/api/v1
  gateway   https://localhost:8444/v1      OpenAI / Anthropic / Gemini
clients
  ingest    https://localhost:8445          SDK (sibyl.instrument(),
sibyl-run)
Admin login (bundled dex): you@acme.example
  password: make password
```

Day-to-day: `make status`, `make logs`, `make down` (keeps data), `make down PURGE=1` (deletes the database, blobs and Caddy's CA; secrets in `env-real/` stay).

2. TRUST THE LOCAL CERTIFICATE AUTHORITY

By default Caddy issues certificates from its own local CA. Export the root once:

```
make ca                    # writes ./sibyl-local-root.crt
```

Then either add it to your OS trust store (macOS: `sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain sibyl-local-root.crt`; Ubuntu: copy it to `/usr/local/share/ca-certificates/sibyl-local-root.crt` and run `sudo update-ca-certificates`), or pass it per tool (`curl --cacert`, `SSL_CERT_FILE` for the agent process). With ACME or your own certificate (see TLS certificates) this step disappears.

3. SIGN IN

Open the console URL, enter the workspace slug, and sign in through dex with the admin email and the password from `make password` (printed only when you ask; stored in `env-real/demo-password.env`, mode 0600). The admin was created as an invitation; the first sign-in with a verified matching email binds it. You land on **Fleet**, empty, as **Admin**.

Useful for the API examples below: copy the `__Host-sibyl_session` cookie from the browser's developer tools and `export ADMIN="sibyl_sess_<cookie value>"`, plus `export SIBYL=https://localhost:8443 CA=$PWD/sibyl-local-root.crt`.

4. CREATE KEYS FOR YOUR AGENTS

Keys look like `sbl_live_<prefix>_<secret>`; the secret is shown once. From the command line:

```
make key NAME=support-bot                       # SDK key: ingest:write
routing:read handoff:write actions:hold
make key NAME=support-bot KIND=gateway          # gateway key:
gateway:proxy routing:read actions:hold
make key NAME=support-bot AGENT=support-bot     # bound to one registered
agent (recommended)
make key NAME=support-bot DAYS=90               # expiry (default 365 days,
at most 400)
```

or in the console under **Settings · API keys**. Store the secret in your secret manager. A key becomes valid at the gateway within one policy-snapshot poll (about 5 seconds). Rotate and revoke from the console, or `docker compose ... run --rm api python -m sibyl_control.cli keys revoke <tenant> <prefix> --reason "..."`.

The SDK refuses keys that carry `gateway:proxy`, so an agent that uses both paths gets one key of each kind.

5. CONNECT A REAL AGENT

There are three ways in, and they combine.

(a) The gateway: change the base URL. The agent keeps its own provider key, which is passed through to the provider and never stored. Register the agent and its primary model first (step 5d), otherwise the gateway answers `404 sibyl_agent_unknown`.

```
# OpenAI (and any OpenAI-compatible client)
from openai import OpenAI
client = OpenAI(
    base_url="https://localhost:8444/v1",
    api_key=os.environ["OPENAI_API_KEY"],                      # your
provider key
    default_headers={"X-Sibyl-Key": os.environ["SIBYL_GATEWAY_KEY"], "X-
Sibyl-Agent": "support-bot"},
)

# Anthropic: the native Messages API
import anthropic
client = anthropic.Anthropic(
    base_url="https://localhost:8444",
    api_key=os.environ["ANTHROPIC_API_KEY"],
    default_headers={"X-Sibyl-Key": os.environ["SIBYL_GATEWAY_KEY"], "X-
Sibyl-Agent": "support-bot"},
)

# Gemini: native generateContent through google-genai
from google import genai
from google.genai import types
client = genai.Client(
    api_key=os.environ["GEMINI_API_KEY"],
    http_options=types.HttpOptions(
        base_url="https://localhost:8444",
        headers={"X-Sibyl-Key": os.environ["SIBYL_GATEWAY_KEY"], "X-
Sibyl-Agent": "support-bot"},
    ),
)
```

Run the agent with `SSL_CERT_FILE=$PWD/sibyl-local-root.crt` (or trust the CA system-wide). Each response carries `X-Sibyl-Route`, `X-Sibyl-Provider`, `X-Sibyl-Routing-State`; refusals made by Sibyl carry `X-Sibyl-Error` and `X-Sibyl-Origin: gateway`. Fallback stays within one protocol (an Anthropic agent fails over only to Anthropic-protocol targets).

(b) The SDK: one line.

```
# the SDK is not on PyPI yet; install it into your agent's environment
from this repository
uv pip install ./packages/sibyl-schemas "./packages/sibyl-sdk[openai]"
# or pip install ...
```

```
import sibyl
sibyl.instrument()   # reads SIBYL_*; instruments OpenAI, Anthropic,
Gemini, LangChain/LangGraph,
                     # OpenAI Agents SDK and CrewAI wherever they are
installed
```

```
export SIBYL_API_KEY=sbl_live_...          # the SDK key from step 4
export SIBYL_ENDPOINT=https://localhost:8445
export SIBYL_API_URL=https://localhost:8443
export SIBYL_AGENT_ID=support-bot
```

(c) Zero code: `sibyl-run`. Same environment, no change to the program:

```
env SSL_CERT_FILE=$PWD/sibyl-local-root.crt sibyl-run app.py --your --
args
env SSL_CERT_FILE=$PWD/sibyl-local-root.crt sibyl-run -m my_service.main
```

`sibyl-run` never stops the program from starting: configuration problems go to stderr (never the key) and the program runs unmonitored. `SIBYL_DISABLED=1` turns it off.

An agent Sibyl has not seen before appears within seconds under **Discovered agents** as `unclaimed/<hash>`. It is provisional: monitored, never routed or failed over, until an admin confirms it (**Confirm**, `POST /agents/{id}/claim`) and names it. Bind the agent's keys to it (`AGENT=`) once it exists.

(d) Register providers, models and agents (needed for the gateway and for failover chains), in the console (**Agents**; **Onboarding** walks through a first agent) or through the API:

```
api() { curl -sS --cacert "$CA" -X "$1" "$SIBYL/api/v1$2" -H
"Authorization: Bearer $ADMIN" \
  -H "Idempotency-Key: $(uuidgen)" -H 'Content-Type: application/json'
${3:+-d "$3"}; }
PROVIDER=$(api POST /providers
'{"slug":"openai","name":"OpenAI","provider_kind":"openai",
  "dialect":"openai_compatible","base_url":"https://api.openai.com/v1"}'
| jq -r .id)
MODEL=$(api POST /models "
{\"provider_id\":\"$PROVIDER\",\"model_name\":\"gpt-4o-mini\"}" | jq -r
.id)
api POST /agents "{\"slug\":\"support-bot\",\"name\":\"Support
bot\",\"kind\":\"chatbot\",\"tier\":3,
  \"fail_mode\":\"open\",\"version\":{\"primary_model_id\":\"$MODEL\"}}"
| jq '{id, slug, lifecycle}'
```

Dialects are `openai_compatible`, `anthropic` (base URL `https://api.anthropic.com`) and `gemini` (`https://generativelanguage.googleapis.com`). Provider base URLs must be public `https` hosts. Continuity plans and fallback chains (what happens when the provider fails) are set per agent in the console; see operations.

6. WRITE A FIRST ACTION GATE POLICY

A policy decides each proposed tool call before it runs: `allow`, `block`, or `escalate` to a person. In the console: **Action policies** → **New**, edit, **Simulate**, **Approve**. Or through the API:

```
cat > first-policy.json <<'EOF'
{
  "name": "Refunds and email guard",
  "scope_kind": "tenant",
  "reason": "First Action Gate policy for the support agents",
  "document": {
    "schema": "sibyl-action-policy/1",
    "default_effect": "allow",
    "sensitive_tools": ["payments.*"],
    "external_tools": ["email.send"],
    "hold_timeout_s": 900,
    "rules": [
      {"rule_id": "refund-limit", "tool": "payments.refund",
       "when": {"gt": ["/amount", 500]}, "effect": "escalate",
       "reason": "Refunds above 500 need a person"},
      {"rule_id": "no-card-numbers-out", "tool": "email.send",
       "data_class": {"secrets": true, "pii": ["credit_card"]}, "effect":
"block",
       "reason": "Card numbers and secrets never leave by email"}
    ]
  }
}
EOF
POLICY=$(api POST /action-policies "$(cat first-policy.json)" | jq -r
.id)
api POST /action-policies/$POLICY/simulate
'{"tool":"payments.refund","args":{"amount":900}}' | jq '{effect,
rule_id, reason}'
# {"effect": "escalate", "rule_id": "refund-limit", "reason": "Refunds
above 500 need a person"}
api POST /action-policies/$POLICY/approve '{"reason":"Reviewed with the
support lead"}' | jq '{id, state}'
```

The approved policy is signed into the next snapshot and enforced by both the gateway (tool calls in model responses) and the SDK (`@sibyl.tool`, LangChain, OpenAI Agents SDK and CrewAI tools). Escalated actions wait under **Held actions**; the requester can never approve their own. The policy language is in the technical document.

7. REGISTER AN EXTERNAL MONITOR

An outside agent (your own Claude, OpenAI or Gemini agent, an MCP-capable assistant, an Agent SDK app, or a plain script) can watch the fleet and report findings:

1. **Register** it (console **Monitors** → **Register**, or `POST /api/v1/monitors` with a name, kind `mcp` | `webhook` | `api`, owner and the agent groups it may watch). Registration issues a key bound to the monitor with `monitor:read` (fleet, health, runs, drift, alerts, incidents, held actions, within the watched groups, metadata only) and `findings:write`. The key is shown once. (`make key` does not mint monitor keys; they always belong to a registered monitor.)
2. **Observe** through the REST API, the **Sibyl MCP server** (`sibyl-mcp`, stdio or streamable HTTP; tools such as `list_agents`, `get_health`, `list_runs`, `get_drift`, `list_alerts`, `list_held_actions`, `submit_finding`, `recommend_on_held_action`), or **signed webhooks** (HMAC-SHA256, timestamped, retried) for alert, incident, held-action, drift and run events.
3. **Report** with `POST /api/v1/findings` (`source: external`). Findings are advisory, like Sentinel's: they can open watch-only alerts and attach recommendations, never allow, block, approve, fail over or change policy.

Ready-to-run monitors for Claude (MCP and the Agent SDK), the OpenAI Agents SDK, Gemini, a webhook receiver and a plain script are in `examples/monitors/`. Install the MCP server from a checkout of this repo (the packages are not on PyPI):

```
pip install ./packages/sibyl-schemas ./packages/sibyl-sdk
./packages/sibyl-mcp
export SIBYL_API_URL=https://localhost:8443 SIBYL_MONITOR_KEY=<monitor
key>
claude mcp add sibyl --env SIBYL_API_URL="$SIBYL_API_URL" --env
SIBYL_MONITOR_KEY="$SIBYL_MONITOR_KEY" -- sibyl-mcp
```

Webhook deliveries are signed `Sibyl-Signature: t=<unix>,v1=<hex HMAC-SHA256(secret, "<t>." + body)>`; verify them with `sibyl.monitor.verify_webhook` or the receiver example.

8. ENABLE SENTINEL

Sentinel is the built-in advisory monitor. It runs on your own model key, within a daily budget: **Settings** → **Sentinel** (`/settings/sentinel`), choose the provider (Anthropic, OpenAI or Gemini) and model, paste the key (write-only: it is sealed under the tenant's credential key and never shown again), set the daily budget in USD and the sampling rate, **Test**, then **Enable**. Saving asks you to confirm it is you (step-up). Findings appear next to incidents, held actions and drift, and people rate them. The worker calls the provider from inside the stack with the container's normal trust store; behind a TLS-inspection proxy that call fails today (see Troubleshooting).

CAPTURE POLICY

The tenant's capture policy (D12) decides what Sibyl stores from prompts and responses: `metadata` (tool names, statuses, timings, counts; no content), `redacted` (content with PII and secrets redacted) or `full` (secrets are still scrubbed). Bootstrap sets `metadata`; change it under **Settings** (capture policy) after your privacy review (payload views, data-class rules on content and Sentinel's evidence need `redacted` or `full`). To make the choice yourself before any key exists, start with `SIBYL_BOOTSTRAP_CAPTURE_MODE=none make up`; keys cannot be minted until a policy exists.

Production hardening

OIDC PROVIDER

The bundled dex holds one local user and is meant for evaluation. For production, use your IdP:

1. Register a confidential web application in the IdP (Okta, Microsoft Entra ID, Google Workspace, Keycloak, Auth0, Ping…): redirect URI `https://<your host>/api/v1/auth/callback`, scopes `openid email profile`, authorization-code flow with PKCE.

2. Put the client secret in a file only you can read, then:

```
export SIBYL_OIDC_ISSUER=https://login.example.com/oauth2/default   # exactly the token's `iss`
export SIBYL_OIDC_CLIENT_ID=0oa1b2c3d4
export SIBYL_OIDC_CLIENT_SECRET_FILE=~/secrets/sibyl-oidc-client-secret
export SIBYL_OIDC_EMAIL_DOMAINS=example.com     # optional: only these domains may sign in
export SIBYL_ADMIN_EMAIL=platform-admin@example.com
make up
```

   dex is then not started. `real.sh` copies the secret into the file secret `oidc_client_secret`.

3. The admin invitation is bound at the first sign-in whose ID token carries the admin's email with `email_verified: true` (Okta, Google, Keycloak, Auth0). Entra ID does not send `email_verified`: set `SIBYL_ADMIN_SUBJECT` to the admin's `sub` claim for this app registration before `make up`.

Further users: invite them (`POST /api/v1/users/invitations` with their email and roles), or allow `SIBYL_OIDC_EMAIL_DOMAINS` and just-in-time roles (`python -m sibyl_control.cli tenant add-issuer ... --domain example.com --jit-role viewer`). Federating dex to an upstream directory (LDAP, SAML, GitHub) is an alternative: edit `deploy/compose/dex/config.real.yaml` (see the dex connector docs).

TLS CERTIFICATES

Mode	Settings	Notes
Local CA (default)	`SIBYL_TLS=internal`	Clients must trust `sibyl-local-root.crt` (`make ca`).
ACME (Let's Encrypt / ZeroSSL)	`SIBYL_SITE_HOST=sibyl.example.com`, `SIBYL_TLS=ops@example.com`, `SIBYL_PORT_HTTPS=443`, `SIBYL_BIND_ADDRESS=0.0.0.0`	Needs a public DNS name pointing at the host and port 443 reachable from the internet (the TLS-ALPN challenge; port 80 is not published). Gateway and ingest share the certificate on 8444/8445.
Your own certificate	`SIBYL_TLS="/certs/fullchain.pem /certs/privkey.pem"`, `SIBYL_TLS_CERT_DIR=/etc/sibyl/tls`	The directory is mounted read-only at `/certs`. Renew by replacing the files and `docker compose ... restart caddy`.

`SIBYL_PUBLIC_URL` defaults to `https://SIBYL_SITE_HOST[:SIBYL_PORT_HTTPS]`; set it explicitly when a load balancer or proxy in front changes the origin. The OIDC redirect URI and the console's CSRF origin follow it. **Behind a load balancer**: terminate TLS at Caddy (TCP passthrough), or accept that the per-client rate limits and audit IP hashes see the balancer's address. The ACME mode is documented but has not been exercised end to end in this repository's CI (it needs a public name).

SECRETS AND ROTATION

- All secrets live in `deploy/compose/env-real/secrets/` (directory 0700), never in env files, images or the repository; each service receives only its own (the matrix is at the top of `deploy/compose/docker-compose.yml`). Back up this directory **separately** from database backups.
- Escrow offline, in two places: `kek` (losing it destroys every tenant's data), `key_pepper`, `ledger_signing_key`, `anchor_signing_key`, `snapshot_signing_key`.
- Rotation procedures (database role passwords, API keys, signing keys, the KEK): ops/runbooks/rotate-keys.md. After rotating a role password, re-run `make up` (it re-applies role passwords through `bootstrap`).
- `deploy/compose/init-env.sh --force` regenerates everything and makes existing data unreadable; never use it on a deployment you want to keep.

BACKUPS AND RESTORE DRILL

Backups are encrypted (age), signed logical dumps with the erasure-ledger head recorded in a manifest: `ops/backup/backup.sh` and `ops/backup/restore.sh`, procedure in ops/runbooks/backup-restore.md. For the real deployment, the paths differ from the runbook's demo defaults:

- Postgres: `127.0.0.1:${SIBYL_PORT_PG}` (5435), user `postgres`, password in `deploy/compose/env-real/secrets/postgres_password` (put it in `~/.pgpass`, mode 0600).
- Ledger public key: `deploy/compose/env-real/public/ledger_signing.pem`.
- Blob store (ledger and payloads): the `sibyl-real_blobstore` volume (`docker compose -p sibyl-real ... cp worker:/var/lib/sibyl/blob/ledger <dir>`).
- Backup target: a directory on another disk or host (`--offhost`), never next to the KEK; schedule it daily (cron or a systemd timer) and keep `--retention-days` aligned with the tenant's backup retention.

Run the restore drill in the runbook before go-live and quarterly: restore into a new database, re-apply the erasure ledger, verify the audit chain (`sibyl audit verify`), then swap. Point-in-time recovery (pgBackRest) is planned.

UPGRADES AND MIGRATIONS

```
git fetch --tags && git checkout v1.1.0      # or: export
SIBYL_IMAGE_TAG=1.1.0 (take a backup first: ops/backup/backup.sh)
make up                                      # rebuilds or pulls, runs
migrate, restarts what changed
```

Migrations are forward-only and run as the non-superuser `sibyl_migrate` role before any control-plane process starts; the gateway keeps serving from its last signed snapshot meanwhile. Read the CHANGELOG for each version first. To roll back, restore the pre-upgrade backup and the previous tag.

RESOURCE SIZING

Fleet	Host	Notes
up to 50 agents, a few requests/s	4 vCPU, 8 GB, 50 GB SSD	the defaults
up to 500 agents, about 100 requests/s through the gateway	8 vCPU, 16-32 GB, 200 GB SSD	raise Postgres `shm_size` and memory; consider S3-compatible blob storage (`SIBYL_BLOB_BACKEND=s3`)
more		split Postgres onto a managed instance and run several gateways; talk to us

Payload capture (`redacted` / `full`) and retention drive disk use far more than agent count.

PRODUCTION CHECKLIST

- ☐ Own OIDC issuer, admin bound, dex not running (`make status`)
- ☐ ACME or own certificate; `SIBYL_BIND_ADDRESS` only as wide as needed; firewall to 443/8444/8445
- ☐ Secrets directory backed up separately; KEK and signing seeds escrowed offline
- ☐ Daily encrypted backups off-host; restore drill passed
- ☐ Capture policy chosen after a privacy review

- ☐ Release images pinned (`SIBYL_IMAGE_TAG`), not local builds
- ☐ No external security assessment yet (D18): treat as pre-production until one is done

Codespaces

Demo: Open in Codespaces (demo). The codespace installs and builds the console, runs `make demo` and opens port 3000 in the browser. GitHub's port forwarding rewrites the `Host` header to `localhost:3000`, which the console's CSRF check would reject; `ops/demo/console.sh` detects the codespace and puts a small origin proxy (`ops/demo/origin-proxy.mjs`) in front that restores the public origin.

Real: Open in Codespaces (real). Choose a 4-core machine or larger. The codespace installs Docker-in-Docker, uv and Node, then runs `.devcontainer/real/start.sh`, which sets `SIBYL_PUBLIC_URL=https://<codespace>-8443.app.github.dev` and runs `make up` (about 10 minutes the first time). It prints the console URL and the admin login; `make password` shows the password.

How sign-in works there: the OIDC issuer and the redirect URI are both under the codespace URL, so the browser stays on one origin; Caddy serves TLS to GitHub's forwarder (port protocol `https`) and restores the public origin for the console. Limitations:

- Forwarded ports are private to you by default (GitHub signs you in). Agents outside the codespace reach the gateway and ingest only if you make ports 8444/8445 **public** in the Ports tab (anyone with the URL can then reach the endpoints; Sibyl keys still apply), or tunnel them with `gh codespace ports forward 8444:8444 8445:8445` and use `https://localhost:8444` with the codespace's CA (`make ca`). Agents running inside the codespace use `https://localhost:8444` directly.
- The codespace URL changes when you create a new codespace; the `make up` in `start.sh` follows it, but sessions from the old URL end.
- Codespaces stop after inactivity; data persists in the codespace's Docker volumes until the codespace is deleted. It is an evaluation environment, not a production host.

Troubleshooting

Symptom	Cause and fix
`port 8443 (SIBYL_PORT_HTTPS) is in use`	Another stack (often `make demo-stack`) holds it. `make demo-stack-down`, or set `SIBYL_PORT_HTTPS=9443` etc. and re-run `make up`. Find the holder: `lsof -iTCP:8443 -sTCP:LISTEN`.
`Pool overlaps with other one on this address space`	Another Docker network uses the edge subnet. Set `SIBYL_EDGE_SUBNET`, `SIBYL_CADDY_IP`, `SIBYL_CONSOLE_IP`, `SIBYL_EDGE_IP_RANGE` to a free /24 (all four consistent).
`Docker Compose >= 2.24.4 is required`	Update Docker Desktop, or the `docker-compose-plugin` package.
`no Docker daemon`	Start Docker Desktop, or `colima start`.
Colima: `invalid mount config ... bind source path does not exist` / secret files missing	The repository is outside `$HOME`; move it under your home directory.
Colima: builds very slow or run out of memory	`colima stop && colima start --cpu 4 --memory 8`.
Build fails with `UnknownIssuer` / `self-signed certificate`	TLS-inspection proxy: set `SIBYL_EXTRA_CA_FILE` (see above).
Browser: `NET::ERR_CERT_AUTHORITY_INVALID`	Trust `sibyl-local-root.crt` (`make ca`), or use ACME / your own certificate. After `make down PURGE=1` the CA is new: export and trust it again.
Python agent: `CERTIFICATE_VERIFY_FAILED` to localhost	Run it with `SSL_CERT_FILE=$PWD/sibyl-local-root.crt`.

Symptom	Cause and fix
Gateway: `502 upstream_error` for a real provider	The gateway cannot reach the provider; behind a TLS-inspection proxy set `SIBYL_UPSTREAM_CA_HOST_FILE` and `make up`.
Gateway: `401 sibyl_key_invalid` right after `make key`	Wait for the next snapshot poll (about 5 s). Otherwise the key is revoked, expired or from another deployment.
Gateway: `404 sibyl_agent_unknown`	Register the agent with a primary model (step 5d) or confirm the discovered one.
Ingest: `403 capture_policy_missing`	Set the tenant capture policy (Settings, or Onboarding).
Sentinel test fails behind a TLS-inspection proxy	The worker has no setting for an extra CA yet (open issue); allow-list the provider host in the proxy, or run Sentinel from a network without inspection.
Sign-in returns to the login page with “Sign-in failed”	The admin email differs from the one in the ID token, the IdP does not send `email_verified` (set `SIBYL_ADMIN_SUBJECT`), or `SIBYL_OIDC_ISSUER` does not equal the token’s `iss` exactly (trailing slash!). `make logs` shows `access.login_failed` with a cause in the audit log.
`migrate` fails: `bootstrap (0000) has not run`	Run `make up` (it runs bootstrap first), not `docker compose up` alone.
Migrations fail after an upgrade	`make logs` (service `migrate`); fix the cause and re-run `make up`. Never edit applied migrations; restore from backup if the database is inconsistent.
A service stays `unhealthy`	`make logs`, or `docker compose -p sibyl-real -f deploy/compose/docker-compose.yml -`

Symptom	Cause and fix
	`f deploy/compose/docker-compose.real.yml --env-file deploy/compose/.env.real logs <service>`; each process names missing or invalid settings at start.
Start from scratch	`make down PURGE=1 && rm -rf deploy/compose/env-real deploy/compose/.env.real && make up` (destroys all data and secrets).

Reference

File	Purpose
`Makefile`	`demo`, `demo-stack`, `up`, `down`, `logs`, `status`, `key`, `password`, `ca`
`deploy/compose/real.sh`	the real deployment's commands (`--help`)
`deploy/compose/real.env.example`	every real-deployment setting, documented
`deploy/compose/docker-compose.yml`	the base stack and its secret matrix
`deploy/compose/docker-compose.real.yml`	real-deployment override (no simulator, configurable OIDC/TLS/binding)
`deploy/compose/docker-compose.images.yml`	use GHCR release images (`SIBYL_IMAGE_TAG`)
`deploy/compose/caddy/Caddyfile.real`, `dex/config.real.yaml`	TLS and the bundled IdP for the real deployment
`deploy/compose/real/bootstrap.py`	first-run tenant, admin, capture policy and gateway keys
`ops/demo/console.sh`	`make demo`
`ops/runbooks/`	backup and restore, key rotation, incidents, provider outages
`docs/pdf/`	product, technical, technical thesis and sales documents